Session 06 — Combined Homework

Session 05 Components + Session 06 State/Events Practice

Complete before: Session 07 **Time you need:** about 60–75 minutes for CORE (two parts, described below)

Why this homework is combined

Session 05's own planned homework (component practice) was **not** handed out the night it was written. Instead of skipping that practice, it is combined here with Session 06's own planned homework (state/events practice) into one assignment. You are not missing anything — this one document contains both.

Every exercise below only uses **already-taught** concepts — nothing here previews Session 07.

No `.map()`. No arrow functions. No type aliases. No string-literal unions. No collection state. No callback props. No spread. No `useEffect`. No destructuring. Nothing you write tonight needs any of these.

What you already have

Start from your own Session 06 End project — the one you finished tonight in class and teamwork (filters, controlled search, Show/Hide Tasks). Confirm it runs with `npm run dev` and shows no console errors before you begin.

If your project is not working, ask the instructor for the clean Session 06 End reference before starting the homework.

Before you start the homework

Homework happens directly on your own personal branch — the same branch you already use for class work. There is no separate homework branch.

```
git checkout <your-student-branch>
git status
```

Confirm you're on your own branch and your tree is clean, then complete the homework below.

Validate:

```
npm run build
```

Commit:

```
git add .
git commit -m "Session 06 combined homework"
```

Push:

```
git push
```

For the complete Git workflow — checking out your branch, getting group updates, resolving conflicts, and pushing — see the Session 05 **Git & GitHub Handbook** shared by your instructor.

CORE — everybody does BOTH parts

PART A — Session 05 component practice: build a `SectionTitle` component

Create `src/components/SectionTitle.tsx`.

Requirements:

- It needs exactly one prop: `title`, a `string`.
- It should render that title as a heading.
- Give it a typed props interface, the same shape as `StatCardProps`.
- It's a normal function, not an arrow function.

Import it into `App.tsx` and use it **once**, directly above your task list, passing whatever title you like (for example `"Your Tasks"`).

Hint, if you're stuck on how to render a heading from a prop: you already know how to put a TypeScript value inside TSX with `{}` — you used it for `StatCard`'s `value`. A heading tag reads its text the same way any other element does.

PART A acceptance criteria

- ☐ `src/components/SectionTitle.tsx` exists with a typed props interface (one required `title: string`).

- □ `SectionTitle` is a normal function, not an arrow function.
- □ Props are read as `props.title` — no destructuring.
- □ `SectionTitle` is used once in `App.tsx`, above the task list.

PART B — Session 06 state/events practice: build a name greeting

Add a new small, self-contained section to `App.tsx` (directly inside it, no new component file needed — though you may create one if you prefer, using the same pattern as `Header` / `StatCard`).

Requirements:

- A controlled text input for a `name`, using `useState("")`, exactly like `searchText`.
- A normal, named `ChangeEvent<HTMLInputElement>`-typed handler that updates the state, exactly like `handleSearchChange`.
- `value` and `onChange` wired on the input, exactly like the search box.
- A paragraph that appears **only when the name is not empty**, using the exact ternary-to-`null` shape already built live for "Searching for: …" — for example, `Hello, {name}!`.

PART B acceptance criteria

- □ A new `useState("")` piece of state exists for the name.
- □ A normal, named function (not an arrow function) handles the change event, typed `ChangeEvent<HTMLInputElement>`.
- □ The input has both `value` and `onChange` wired.
- □ The greeting paragraph only exists in the JSX when the name is not an empty string.

CORE acceptance criteria (both parts)

- □ PART A and PART B are both complete (see above).
- □ `npm run build` succeeds with zero errors.
- □ Nothing else about the app changed.

STRETCH — if CORE was comfortable, extends BOTH parts

PART A — give `SectionTitle` a second required prop

Extend the **same** component with a second **required** prop: `subtitle`, also a `string`. Render it too — a short line under the title.

Update your one usage in `App.tsx` to pass both props.

Do not make `subtitle` optional. Both props stay required, matching how every props interface built in class has worked so far.

PART B — choose the greeting message with ordinary `if` / `else` logic

Extend the name-greeting piece so the message depends on what was typed, using a plain `if` / `else if` / `else` block **before** your `return` statement — not a ternary inside the JSX this time.

Example shape:

```
let greetingMessage = "";

if (name === "") {
    greetingMessage = "";
} else if (name === "admin") {
    greetingMessage = "Welcome back, admin.";
} else {
    greetingMessage = "Hello, " + name + "!";
}
```

Keep your CORE paragraph's existing ternary — the one deciding whether the paragraph exists at all. Don't remove it and don't write a second ternary. The only thing that changes inside that paragraph is which **text** it shows: instead of the greeting text you typed directly into the JSX for CORE, render `{greetingMessage}` there instead — the message the `if` / `else if` / `else` block above just computed.

Two different jobs, staying separate:

- **`if` / `else if` / `else`** → chooses *which message* to show.
- **Your existing ternary** → still decides *whether the paragraph exists at all* (unchanged from CORE).

Do not use a second ternary to choose the message text — that's the part `if` / `else if` / `else` is for.

STRETCH acceptance criteria (both parts)

- □ `SectionTitleProps` now has two required props, and the one usage in `App.tsx` passes both.
- □ A `let greetingMessage` (or similar) is computed with `if` / `else if` / `else` before `return`.
- □ At least one specific name (e.g. `"admin"`) produces a different message than the general case.
- □ The existing CORE ternary deciding whether the greeting paragraph exists is still there, unchanged — only the text rendered inside it changed.
- □ `npm run build` succeeds with zero errors.

CHALLENGE — for students who want more, extends BOTH parts

PART A — build a `PersonSummary` component, used three times

Create `src/components/PersonSummary.tsx`.

Requirements:

- Two required props: a person's `name` (`string`) and their `taskCount` (`number`).
- It should render one line combining both — for example, a name followed by how many tasks they have.

Import it into `App.tsx` and render it **three times, manually**, once per person who owns a task in your app, each with that person's real name and task count.

No `.map()`. Three separate calls, by hand — exactly how you rendered three `StatCard`s and three `TaskItem`s in class.

PART B — add an independent Show/Hide toggle around the greeting mini-feature

Add one more boolean piece of state — the same shape as teamwork's `showTasks` — that shows or hides the **entire** name-input-and-greeting section from CORE/STRETCH.

Requirements:

- `useState(true)` (or `false` — your choice of starting value).
- A normal, named handler using `!` to flip it, exactly like `handleToggleTasks`.
- One button whose text changes based on the state (e.g. "Hide" / "Show"), exactly like teamwork's button.
- The whole input-and-greeting section wrapped in the same ternary-to-`null` shape.

This is a completely independent piece of state from teamwork's `showTasks` — it must not reuse or interfere with it.

CHALLENGE acceptance criteria (both parts)

- □ `src/components/PersonSummary.tsx` exists with a typed props interface (`name: string`, `taskCount: number`), used exactly three times with three different sets of props.
- □ A second, independent `useState(true)` (or `false`) boolean exists for the greeting toggle.
- □ A normal, named handler toggles it with `!`.
- □ One button's text changes based on that boolean.
- □ The entire name-greeting section only renders when that boolean says it should.
- □ `npm run build` succeeds with zero errors.
- □ No `.map()`, arrow functions, type aliases, unions, collection state, callback props, or spread appear anywhere in your solution.

One question to think about — no code required

The task list still doesn't filter by status or search text, even though both pieces of state exist and work.

What would need to exist for clicking "Completed" to actually hide the pending tasks?

Write your answer in a sentence or two. You do not need to implement it — just think about what's missing. That's exactly what next session is about.